DOCUMENTATION TECHNIQUE · V1.0

Bean Beacon — Cahier des charges

Application web de découverte de cafés construite avec React, Express, MongoDB et orchestrée via Docker Compose, avec supervision et déploiement CI/CD GitLab.

React + Vite + TypeScript
Express + TypeScript
MongoDB
Docker Compose
GitLab CI/CD
Prometheus · Grafana · Uptime Kuma

Bean Beacon Project

Documentation finale

Table des matières

§ 01 Présentation du projet 3

§ 02 Architecture 3

§ 03 Prérequis 4

§ 04 Démarrage avec Docker Compose 4

§ 05 Tests et validations 5

§ 06 Workflow Git et GitLab 5

§ 07 Pipeline GitLab CI/CD 6

§ 08 Déploiement local de production 6

§ 09 Supervision 7

§ 10 Livrables finaux 8

§ 11 Problèmes fréquents 8

§ 01

Présentation du projet

Bean Beacon est une application web qui permet de découvrir des cafés, de rechercher des établissements à proximité, d'ajouter des favoris et de publier des évaluations. L'architecture existante est conservée — l'exemple Flask des guides n'est pas utilisé.

Frontend
Interface utilisateur réactive
`React + Vite + TypeScript`

Backend
API REST & logique métier
`Express + TypeScript`

Base de données
Stockage persistant
`MongoDB :27017`

Conteneurs
Orchestration complète
`Docker Compose`

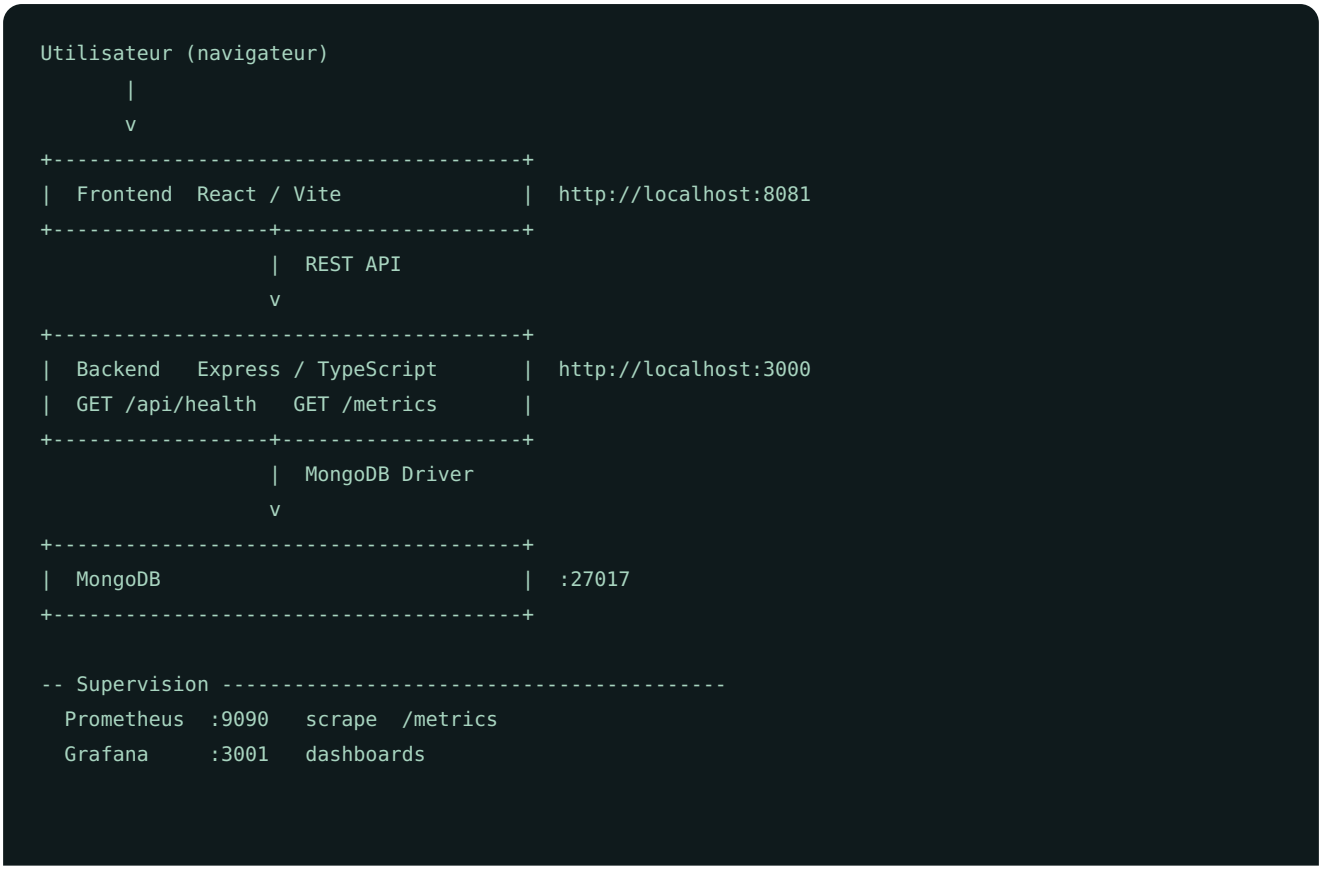
CI/CD
Automatisation du déploiement
`GitLab Pipeline`

Supervision
Monitoring & alertes
`Prometheus · Grafana`

§ 02

Architecture

L'ensemble des services communique sur un réseau Docker Bridge partagé. Les flux applicatifs traversent trois couches : frontend → backend → base de données.



```
Uptime Kuma :3002   moniteur HTTP /api/health
```

Service	Adresse locale	Rôle
Application	`http://localhost:8081`	Interface utilisateur
API	`http://localhost:3000`	Endpoints REST
Health check	`http://localhost:3000/api/health`	Disponibilité de l'API
Métriques	`http://localhost:3000/metrics`	Prometheus scrape
Prometheus	`http://localhost:9090`	Collecte de métriques
Grafana	`http://localhost:3001`	Dashboards
Uptime Kuma	`http://localhost:3002`	Moniteur de disponibilité

§ 03

Prérequis

Docker Desktop avec Docker Compose — orchestration des conteneurs

Git — versioning et intégration GitLab CI/CD

Node.js + npm — validations hors Docker (build, lint, tests)

Python + pyinfra — déploiement local automatisé

PowerShell — vérification des outils

```
docker --version
docker compose version
git --version
node --version
npm --version
```

§ 04

Démarrage avec Docker Compose

Depuis la racine du projet, créer le réseau externe si absent, puis démarrer la stack de développement.

PowerShell — démarrage

```powershell
# Se placer dans le projet
cd "C:\Users\msi\Desktop\coffee-quest\bean-beacon"

# Créer le réseau si nécessaire
docker network inspect bean-beacon_app-network *> $null
if ($LASTEXITCODE -ne 0) {
    docker network create bean-beacon_app-network
}

# Valider la configuration puis démarrer
docker compose config --quiet
docker compose up -d --build
```

PowerShell — vérification & test API

```powershell
# État des conteneurs
docker compose ps

# Test de l'API
Invoke-RestMethod "http://localhost:3000/api/health"
# Réponse attendue : { "success": true, "message": "Bean Beacon API is running" }

# Logs en continu
docker compose logs -f backend
docker compose logs -f frontend
```

Arrêt sans perte de données

Utiliser `docker compose stop` pour préserver les volumes MongoDB et Grafana. Éviter `docker compose down -v` qui supprime les données persistantes.

§ 05

Tests et validations

PowerShell — validation backend

```
cd bean-beacon-backend
npm ci
npm run build
npm test -- run
```

PowerShell — validation frontend

```
cd ..\bean-beacon-frontend
npm ci
npm run lint
npm run typecheck
npm run build
cd ..
```

§ 06

Workflow Git et GitLab

La branche `main` doit rester déployable en permanence. Toute modification passe par une branche courte et une Merge Request avec revue humaine avant fusion.

Git — cycle de contribution

```
git checkout -b feature/nom-de-la-fonctionnalite
git add .
git commit -m "feat: description de la modification"
git push origin feature/nom-de-la-fonctionnalite

# Créer une Merge Request sur GitLab
# Revue de code, puis fusion dans main
# Le pipeline CI/CD se declenche automatiquement
```

Revue humaine obligatoire

Le script `demo.ps1` ne crée pas de commit ni de Merge Request automatiquement. Ces actions sont réservées au workflow GitLab afin de garantir une validation manuelle à chaque changement.

§ 07

Pipeline GitLab CI/ CD

Le fichier `.gitlab-ci.yml` orchestre quatre étapes séquentielles. Le Runner doit porter le tag `bean-beacon`. Harbor est exclu du périmètre.

ÉTAPE 01
validate
- Compilation backend
- Lint frontend
- TypeScript check
- Build frontend

→

ÉTAPE 02
test
- Tests unitaires
- Vitest (backend)

→

ÉTAPE 03
security
- Audits npm
- Gitleaks secrets

→

ÉTAPE 04
build
- Image backend
- Image frontend

Tag du Runner requis

Le GitLab Runner doit être enregistré avec le tag exact `bean-beacon`. Harbor est volontairement exclu — les images sont construites sans publication vers un registre privé.

§ 08

Déploiement local de production

PowerShell — déploiement production

```
# Définir le secret JWT (obligatoire)
$env:JWT_SECRET = "remplacer-par-un-secret-long-et-aleatoire"

# Démarrer la version production
docker compose -f docker-compose.prod.yml up -d --build

# Ou via pyinfra
pyinfra @local deploy.py

# Application accessible sur http://localhost
```

Secret JWT obligatoire

La variable `JWT_SECRET` doit être définie avant tout démarrage en production. Utiliser une chaîne longue, aléatoire et gardée secrète.

§ 09

Supervision

Trois outils forment la couche d'observabilité de Bean Beacon. Ils tournent dans le même réseau Compose et communiquent via les noms de services Docker.

Prometheus

Collecte les métriques du backend via scrape sur `/metrics`

`localhost:9090`

Grafana

Dashboards de visualisation. Données dans le volume `grafana_data`

`localhost:3001`

Uptime Kuma

Moniteur HTTP sur `/api/health` — intervalle 60 s, 2 retries

`localhost:3002`

Dans Uptime Kuma, utiliser l'URL interne `http://backend:3000/api/health` (nom de service Docker) plutôt que `localhost`.

PowerShell — test de supervision

```
# Arrêter puis redémarrer le backend
docker compose stop backend
docker compose start backend
# Le moniteur passe en erreur, puis redevient vert.
```

Réinitialisation du mot de passe Grafana

En cas de blocage :

`docker compose exec grafana grafana cli admin reset-admin-password "NouveauMotDePasseFort!"`

§ 10

Livrables finaux

Frontend
bean-beacon-frontend

Backend API
bean-beacon-backend

Base de données
Service mongo (Compose)

Compose dev
docker-compose.yml

Compose prod
docker-compose.prod.yml

Pipeline GitLab
.gitlab-ci.yml

Health check
/api/health

Tests backend
bean-beacon-backend/tests

Déploiement pyinfra
deploy.py

Démo Windows
demo.ps1

Supervision
Prometheus · Grafana · Uptime Kuma

Documentation
docs/FINAL_DOCUMENTATION.md

§ 11

Problèmes fréquents

Aucun conteneur n'apparaît avec `docker compose ps`

La stack n'est pas démarrée. Lancer :

Fix

```
docker compose up -d --build
```

L'API retourne `Route not found` sur `http://localhost:3000/`

Comportement normal : la route racine n'est pas définie. Utiliser `http://localhost:3000/api/health` à la place.

Grafana refuse le mot de passe

Réinitialiser sans supprimer le volume :

Fix

```
docker compose exec grafana grafana cli admin reset-admin-password "NouveauMotDePasseFort!"
```

Préserver les données MongoDB et Grafana

Toujours utiliser `docker compose stop`. La commande `docker compose down -v` supprime définitivement les volumes persistants.